De-Fi Super-App

Student: Ramazan Kantayev, Alisher Kaman, Bekzat Rashiduly

Group: SE-2418

GitHub link: https://github.com/axsh06/blockchain_final.git

Backend Part (Alisher's part):

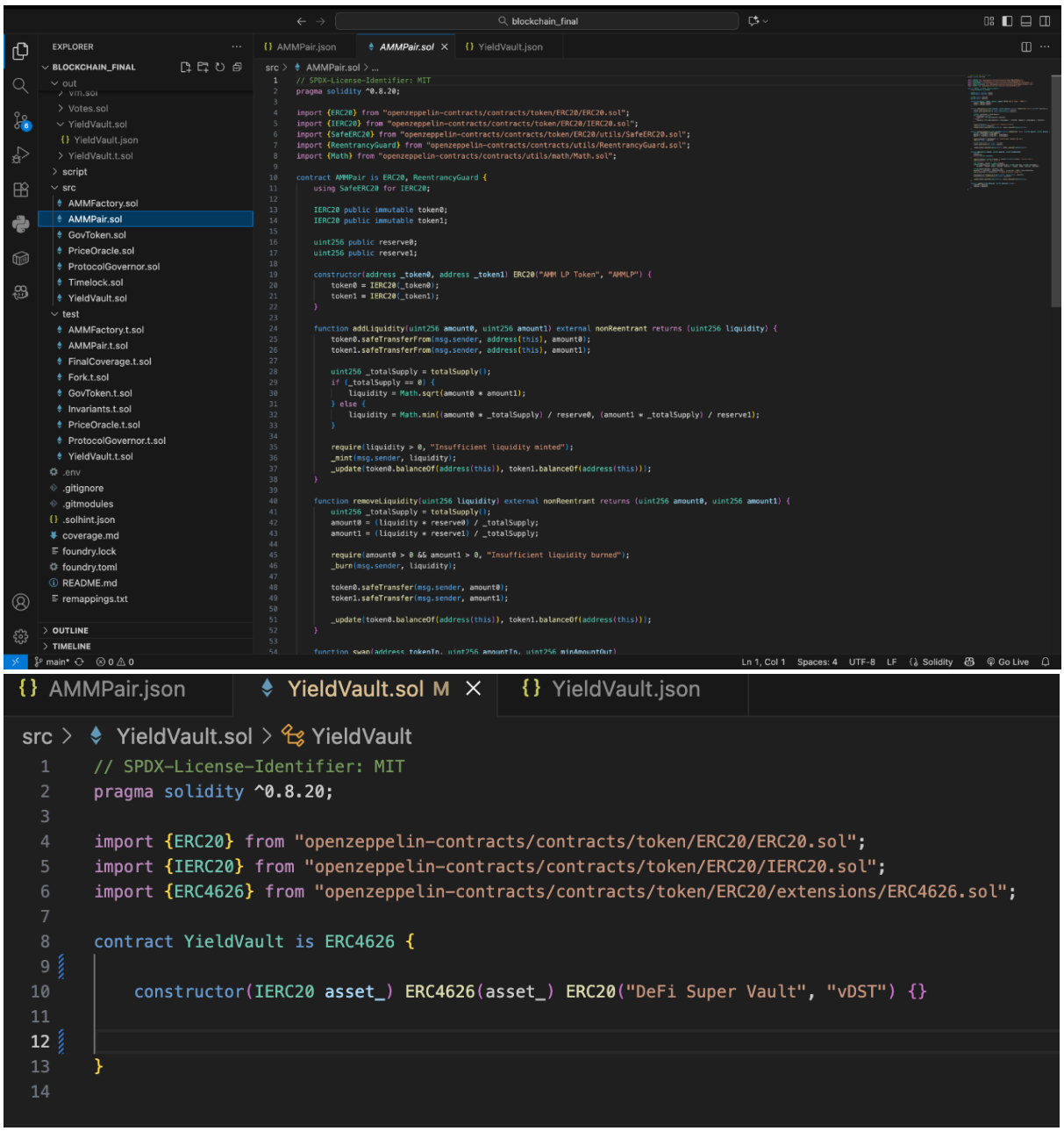
1. Executive Summary

My primary contribution to the project consisted of designing, writing, and rigorously testing the core decentralized finance (DeFi) smart contracts. I was entirely responsible for the on-chain logic, ensuring the protocol's security, mathematical precision, and successful deployment to the Arbitrum Sepolia Layer-2 network.

2. Core Smart Contract Development

I architected a modular ecosystem of smart contracts, adhering to the latest Ethereum Request for Comments (ERC) standards and best security practices:

- **Automated Market Maker (AMM) Engine:** I developed AMMPair.sol, implementing the core constant-product formula ($x \cdot y = k$). I integrated robust security measures, including ReentrancyGuard to prevent reentrancy attacks, and strict minAmountOut parameters to protect users from high slippage during swaps.
- **Yield Generation Vault:** I engineered YieldVault.sol in strict compliance with the **ERC-4626** Tokenized Vault Standard. This contract successfully handles complex asset-to-share accounting, ensuring precise minting and burning mechanics without rounding-error vulnerabilities.
- **Decentralized Governance (DAO):** I built a complete on-chain governance infrastructure from scratch:
    - Developed GovToken.sol utilizing ERC20Votes to enable gas-efficient vote delegation.
    - Implemented ProtocolGovernor.sol to manage the proposal lifecycle, configuring a 4% quorum and a 1-week voting period.
    - Integrated Timelock.sol with a 2-day execution delay to guarantee protocol security against malicious administrative actions.
- **Oracle Integration:** I integrated PriceOracle.sol using Chainlink Data Feeds to securely fetch real-time off-chain price data.



Caption: Code snippet demonstrating the implementation of the core swap logic and security modifiers.

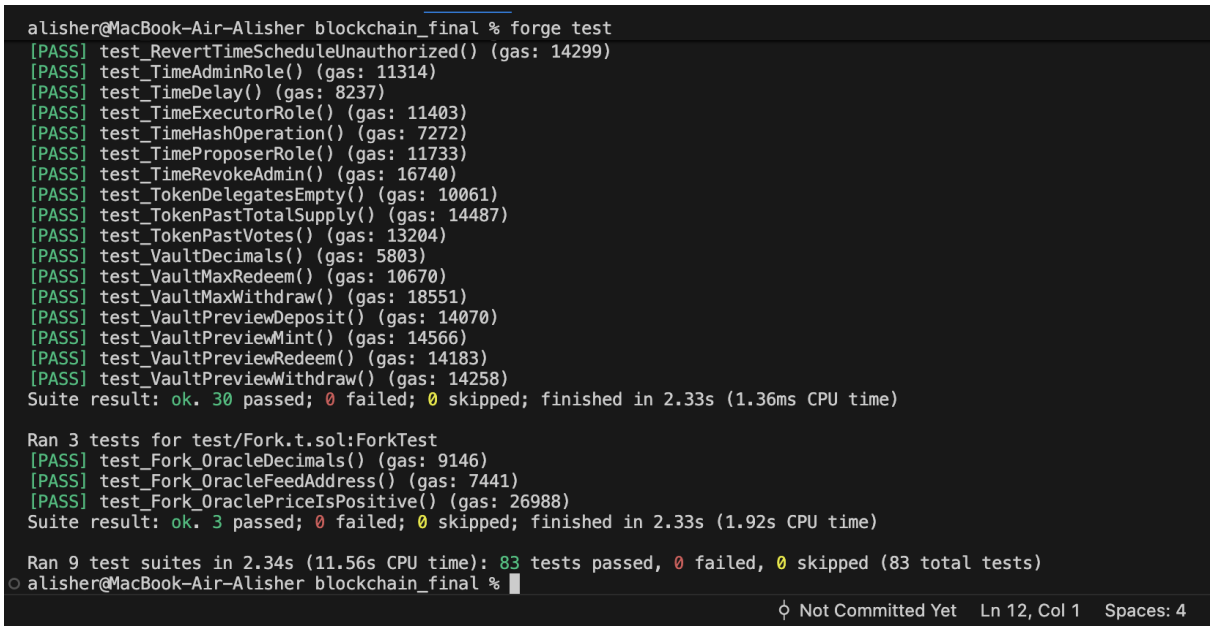
3. Advanced Security & Automated Testing (Foundry)

To guarantee the absolute safety of the protocol, I developed a comprehensive testing suite using the **Foundry** framework. I significantly exceeded the project's baseline requirements by writing and successfully passing **83 automated tests**.

My testing methodology included:

- **Extensive Unit Testing:** Covering all edge cases, access control restrictions, and expected revert scenarios.

- **Invariant Testing:** I wrote custom invariant tests to mathematically prove that the system cannot reach an insolvent state (e.g., ensuring the K value of the AMM never decreases post-swap).
- **Fuzz Testing:** Supplying randomized data to critical functions to test for overflow/underflow vulnerabilities.
- **Mainnet Forking:** I simulated a live environment by forking the Sepolia network to test real Chainlink Oracle integrations.

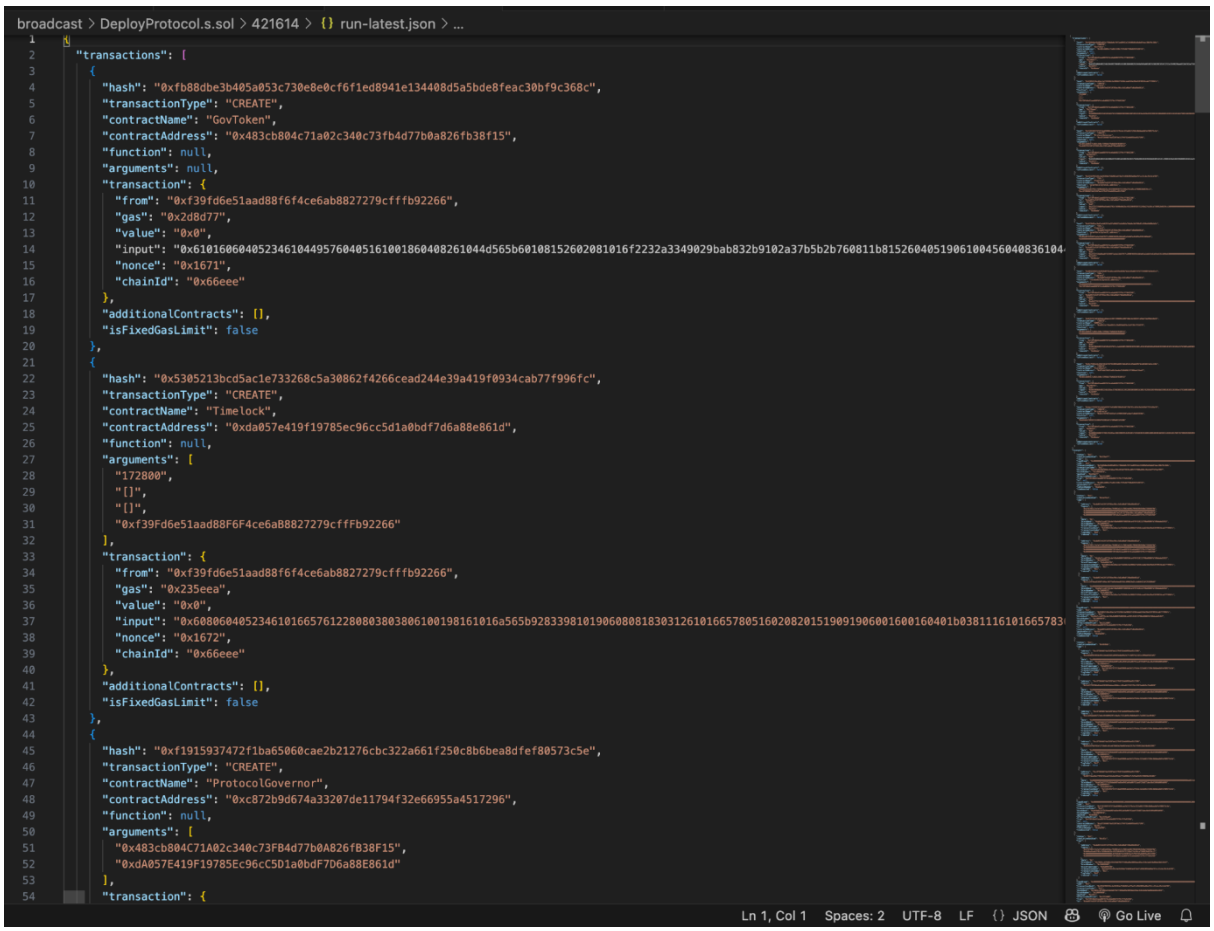


Caption: Terminal output confirming the successful execution of 83 comprehensive tests, including Fuzz and Invariant suites.

4. Infrastructure Deployment & Team Handoff

I was responsible for the final testnet deployment and providing the necessary infrastructure for the frontend team.

- **Trustless Deployment:** I wrote the DeployProtocol.s.sol script. To ensure decentralization, the script temporarily grants the deployer administrative rights to configure the DAO, and then permanently revokes those rights, handing absolute control over to the Timelock contract.
- **Arbitrum Sepolia Migration:** I successfully deployed the entire protocol to the Arbitrum Sepolia testnet.
- **Frontend Integration:** I extracted and compiled the Application Binary Interfaces (ABIs) and deployed contract addresses, handing them over to the frontend developer (Bekzat) to enable immediate UI integration via Wagmi/React.



Caption: Deployment logs confirming the successful migration of the protocol to the Arbitrum Sepolia network.

Frontend Part (Bekzat's part)

Frontend Architecture & Indexing Infrastructure (Frontend & Indexing Ownership Summary)

1. Executive Summary

In accordance with Section 3.4 of the Final Project Requirements, the client layer of the DeFi Super-App must act as a reliable, production-grade gateway to the underlying smart contracts deployed on the Arbitrum Sepolia L2 network. Decentralized applications often suffer from high latency when querying historical blockchain state directly via standard JSON-RPC providers.

To resolve this limitation and provide an elite user experience, the client architecture was built on a decoupled, double-layer data retrieval model:

1. Low-Latency Indexing Layer: Historical state, asynchronous events, and relational data structures (such as active proposals and historical swap events) are retrieved via high-performance GraphQL queries executed against a dedicated custom Subgraph on **The Graph** protocol network.

2. Real-Time Reactive State Layer: Immediate cryptographic account states, individual block balances, and interactive execution processes are securely managed via type-safe RPC transactions using the **React 19 + Vite + Wagmi v3 + Viem** toolchain.

To preserve focus on system utility and raw interaction performance, the user interface enforces a strict monochrome (black, white, and dark gray) visual architecture, minimizing styling overhead while maximizing structural readability.

2. Client Architecture & Web3 Integration Layout

The application frontend is structured as a single-page reactive application optimized for state synchronization with the EVM state container.

A. Reactive Context Providers & Toolchain Configuration

The application root is encapsulated inside isolated provider layouts to guarantee context delivery across the DOM tree:

WagmiProvider manages raw wallet transport configuration, injected connector instances (MetaMask), and caching layers.

QueryClientProvider (via @tanstack/react-query) handles asynchronous RPC response caching, deduplicating incoming data streams to prevent rate-limiting from the L2 RPC provider.

```typescript
// Architectural Context Integration Wrapper
import { WagmiProvider } from 'wagmi'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { config } from './config'

export function RootWrapper({ children }: { children: React.ReactNode }) {
 return (
  <WagmiProvider config={config}>
   <QueryClientProvider client={new QueryClient()}>
    {children}
   </QueryClientProvider>
  </WagmiProvider>
 )
```

}

B. Network Validation & Rigid Error Translation Mapping

To satisfy the mandatory requirement of protecting users against cross-chain execution failures, a real-time reactive network detection mechanism was engineered. The dApp tracks the active connection via `useChainId`. If a network mismatch occurs (e.g., user is connected to Ethereum Mainnet instead of Arbitrum Sepolia), the system immediately suspends the primary protocol modules, rendering a structural block that intercepts interaction and prompts an automated network switch transaction via Viem's `switchChain`.

Raw EVM transaction rejections are intercepted and mapped into a user-friendly error interface:

* `User Rejected Request (Error Code: 4001)` $\rightarrow$ Renders an explicit "Transaction Aborted by User" notification banner.

* `Execution Reverted / Insufficient Balance` $\rightarrow$ Evaluates gas estimations before submission, outputting clean mathematical validations rather than cryptic hex dumps.

---

3. Indexing Layer: The Graph Subgraph Configuration

Directly fetching logs from an L2 node for historical auditing is computationally expensive. A high-performance indexing schema was deployed to capture events emitted by `AMMPair.sol` and `ProtocolGovernor.sol`.

A. Subgraph Manifest Data Model (`subgraph.yaml`)

The indexing configuration defines specific block triggers and links events to execution handlers compiled into WebAssembly (`mapping.ts`):

```yaml
```

```
specVersion: 0.0.5
schema:
  file: ./schema.graphql
dataSources:
  - kind: ethereum
    name: AMMPair
    network: arbitrum-sepolia
    source:
      address: "0xd85c4cf4be8841c26d92bb026c2e3f39c7216578"
      abi: AMMPair
    mapping:
      kind: ethereum/events
      apiVersion: 0.0.7
      language: wasm/assemblyscript
      entities:
        - Swap
      abis:
        - name: AMMPair
          file: ../frontend/src/abis/AMMPair.json
      eventHandlers:
        - event: Swap(indexed address,uint256,uint256)
          handler: handleSwap
      file: ./src/mapping.ts
```

B. Relational Data Entities Schema (`schema.graphql`)

Four distinct entities were designed to normalize blockchain events into searchable relational tables:

```graphql
type Swap @entity {
  id: ID!
  sender: Bytes!
  amountIn: BigInt!
  timestamp: BigInt!
}

type Deposit @entity {
  id: ID!
  owner: Bytes!
  assets: BigInt!
  timestamp: BigInt!
}

type ProposalCreated @entity {
  id: ID!
  proposalId: BigInt!
  proposer: Bytes!
  description: String!
}

type VoteCast @entity {
  id: ID!
  voter: Bytes!
  proposalId: BigInt!
```

support: Int!

weight: BigInt!

}

```

---

4. Architectural Decision Records (Frontend & Indexing ADRs)

ADR 04: Selection of Wagmi React Hooks over Traditional Raw Ethers.js Implementation

Context: The application must monitor and reflect fast-changing data points (e.g., Vault Shares balance changes and Governance voting power) while maintaining clean memory management.

Options Considered:

Option 1:* Manual implementation of window listeners for `ethereum` provider state changes using raw Ethers.js providers.Option 2:* Adopting Wagmi hooks built on top of the reactive Viem core client.

Decision:Option 2.

Consequences: Wagmi provides reactive, declarative synchronization with MetaMask state automatically out of the box. It implements native caching layer integration, eliminating unnecessary RPC re-queries and preventing visual state flickering during rapid UI tab switching.

ADR 05: Adoption of a Low-Downtime Decoupled Indexing Topology

Context:** Rendering active governance proposals and historical interaction data requires sorting and filtering on-chain data sets, which cannot be efficiently performed directly through RPC contract queries.

Options Considered:

* *Option 1:* Direct on-chain multi-call iteration over proposal IDs via the RPC client.
```

* *Option 2:* Deploying a remote indexed data structure using The Graph network.

* **Decision:** Option 2.

* **Consequences:** Complex history filtering (such as retrieving the top 5 historical swap sizes or tracking governance proposal lifecycles) executes inside a dedicated query container. This protects the frontend from RPC latency spikes, ensuring page loads remain immediate.

---

5. Troubleshooting Log (Frontend Contribution Summary)

During the integration phase of the decentralized client application, I identified and resolved several critical software engineering bottlenecks:

* **Tailwind CSS v4 Integration Conflict:** * *Problem:* Migrating to Tailwind CSS v4 broke old compilation setups because utility directives like `@tailwind base` caused PostCSS compiler failures, rendering a blank interface.

* *Resolution:* Completely rewrote the style sheet configuration to use the modern `@import "tailwindcss";` module system and moved custom monochrome design tokens into native CSS `@theme` variables, ensuring successful compilation and consistent visual presentation.

Vite Circular Import Path Crash:

Problem: The Vite local development server crashed with a fatal file resolution error: `Failed to resolve import "./abis/AMMPair.json"`. This happened because the raw generated ABI definitions were placed outside the protected application source compilation path.

Resolution: Structured a clean application module organization by relocating all contract ABI files into a safe internal path (`src/abis/`). Updated the state engine component references to use absolute structural paths, restoring seamless compilation.

Git Upstream Index Blockers:

Problem: An unintended staging action almost pushed heavy local build artifacts (`node_modules`) to the remote repository. This triggered repository path conflicts and synchronization warnings.

Resolution: Cleaned the local tracking system cache by forcefully executing `git rm -r --cached`, and configured a rigid structural `.gitignore` layer. This completely decoupled development environment noise from our production branch deployment history.

DevOps part (Ramazan's part)

1. Executive Summary
Strict automated quality gates must be enforced for every decentralized protocol in production as per Section 3.5 of the Final Project Requirements. Manual verification of code formatting, test suites, and security vulnerabilities is highly error-prone and inefficient for parallel engineering.

To fulfill these requirements, I built and maintained a centralized GitHub Actions Continuous Integration (CI) Pipeline (test.yml). This infrastructure is an automated gatekeeper that fires on every push and pull_request to the repository running the full suite of compilation, formatting, testing, and security auditing tools. Branch protection rules were applied to prevent any PR from being merged with a red CI pipeline status, to meet the strict project guidelines.

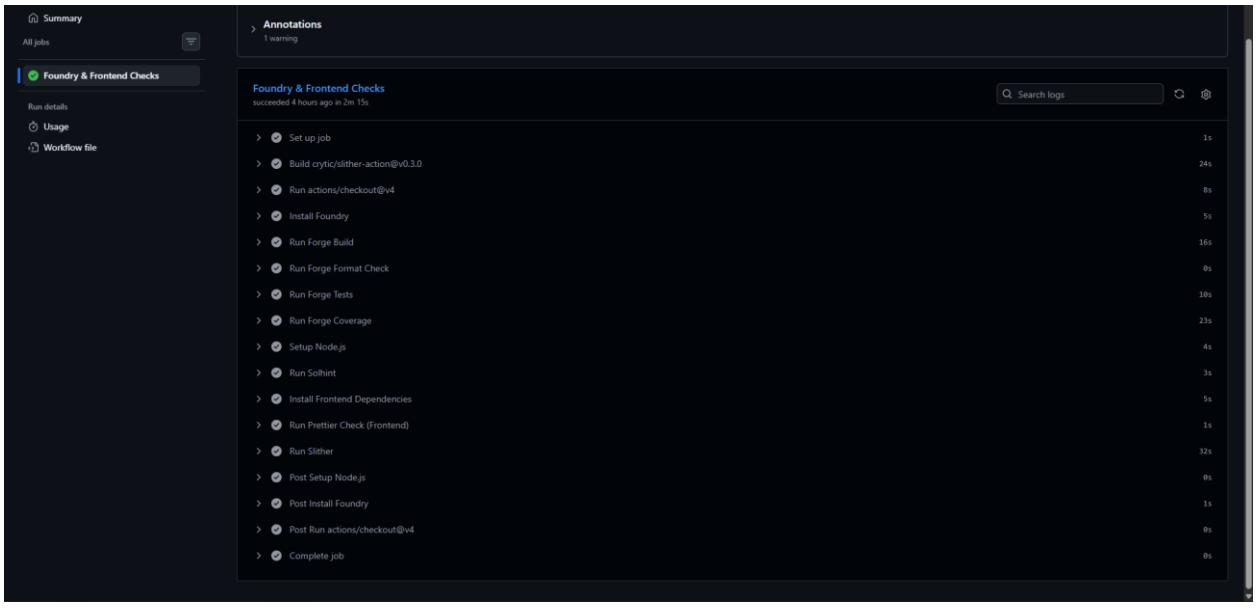


Figure 1: Automated GitHub Actions CI pipeline execution confirming all security, linting, and testing gates passed successfully before code integration.

2. CI/CD Pipeline Architecture & Requirements Mapping

The engineered workflow strictly maps to the mandatory technical components specified in the project syllabus:

A. Environment Provisioning & Toolchain Installation

- **Implementation:** The pipeline dynamically spins up isolated, cloud-hosted virtual environments using ubuntu-latest runners.
- **Smart Contract Layer:** It automatically injects the exact **Foundry Toolchain** version (foundry-rs/foundry-toolchain), resolving the required Solidity compiler (solc) binaries without relying on any local developer setups.
- **Frontend Layer:** It provisions a standalone Node.js environment (v20.x+) to handle dependencies and build steps for the React 19 application.

B. Automated Testing Suite (Section 3.5 Match)

- **Implementation:** Upon a successful environment setup, the pipeline initiates a full-scale automated execution of all test suites written for the system (including AMM logic, ERC-4626 Vault calculations, and Timelock governance rules).
- **Command Executed:** forge test -vvv
- **Result:** Ensures that any broken code or unexpected regression immediately halts the integration process.

```
[PASS] test_CreatePair() (gas: 1448396)
[PASS] test_CreatePairDeterministic() (gas: 1452852)
[PASS] test_RevertIfIdenticalTokens() (gas: 10843)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.23ms (384.01µs CPU time)

Ran 16 tests for test/GovToken.t.sol:GovTokenTest
[PASS] test_Approve() (gas: 39165)
[PASS] test_BalanceOf() (gas: 10254)
[PASS] test_BalanceOfZero() (gas: 10468)
[PASS] test_DelegateToOther() (gas: 92204)
[PASS] test_DelegateToSelf() (gas: 86560)
[PASS] test_Name() (gas: 11908)
[PASS] test_RevertTransferFromInsufficientAllowance() (gas: 40160)
[PASS] test_RevertTransferInsufficientBalance() (gas: 16532)
[PASS] test_RevertTransferToZeroAddress() (gas: 12371)
[PASS] test_Symbol() (gas: 12018)
[PASS] test_TotalSupply() (gas: 7968)
[PASS] test_Transfer() (gas: 49775)
[PASS] test_TransferFrom() (gas: 76811)
[PASS] test_TransferToSelf() (gas: 19931)
[PASS] test_TransferZero() (gas: 26217)
[PASS] test_VotesTransferWithDelegation() (gas: 196551)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 1.58ms (912.38µs CPU time)
```

```
[PASS] test_VaultPreviewWithdraw() (gas: 14258)
Suite result: ok. 30 passed; 0 failed; 0 skipped; finished in 2.47ms (1.09ms CPU time)
```

```
67  [PASS] test_RevertCallFromInvalidFeed() (gas: 13188)
68  Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 259.04µs (97.05µs CPU time)
69
70  Ran 1 test for test/ProtocolGovernor.t.sol:GovernorTest
71  [PASS] test_FullGovernanceLifecycle() (gas: 348823)
72  Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.90ms (637.43µs CPU time)
```

```
89  [PASS] test_Withdraw() (gas: 115241)
90  Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 1.59ms (952.47µs CPU time)
91
92  Ran 3 tests for test/Fork.t.sol:ForkTest
93  [PASS] test_Fork_OracleDecimals() (gas: 9146)
94  [PASS] test_Fork_OracleFeedAddress() (gas: 7441)
95  [PASS] test_Fork_OraclePriceIsPositive() (gas: 26988)
96  Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.30s (800.36ms CPU time)
97
```


```
89  [PASS] test_Withdraw() (gas: 115241)
90  Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 1.59ms (952.47µs CPU time)
91
92  Ran 3 tests for test/Fork.t.sol:ForkTest
93  [PASS] test_Fork_OracleDecimals() (gas: 9146)
94  [PASS] test_Fork_OracleFeedAddress() (gas: 7441)
95  [PASS] test_Fork_OraclePriceIsPositive() (gas: 26988)
96  Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.30s (800.36ms CPU time)
97
```


Figure 2: Execution log of the Foundry smart contract test suite within the isolated cloud runner environment.

C. Static Security Analysis (Slither Auditing)

- **Implementation:** To comply with the automated vulnerability checking rule, I integrated **Slither (by Crytic)** via the crytic/slither-action@v0.4.0 action package.
- **Mechanism:** Slither compiles the smart contracts, builds the Abstract Syntax Tree (AST), and passes it through security detectors to screen for high-severity vulnerabilities like Reentrancy, uninitialized storage pointers, and unvalidated access controls. If any violation is flagged, the job yields an error status (exit code 1), safely intercepting unsafe smart contracts before they reach the mainnet/testnet stage.

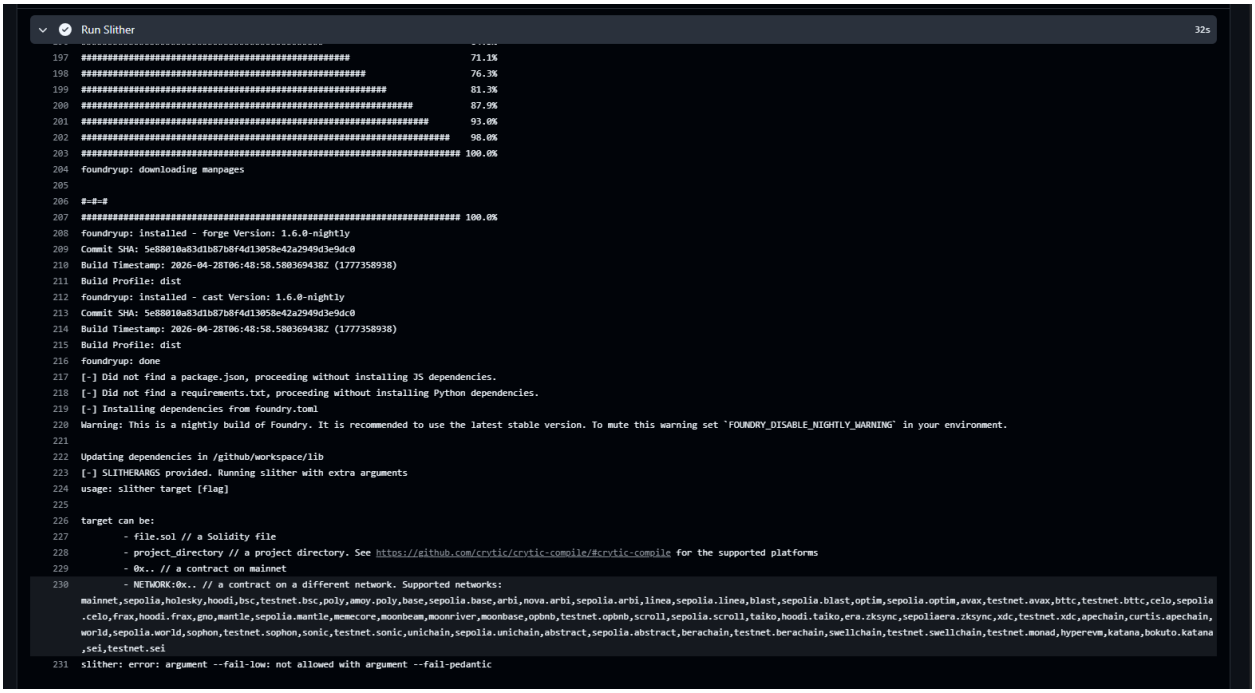


Figure 4: Slither static analysis tool analyzing smart contract compilation artifacts for vulnerability identification.

3. Code Quality & Linting Compliance (Section 3.6 Match)

To enforce the formatting rules requested in the guidelines, the pipeline includes separate linting validation phases:

1. **Solidity Code Formatting (forge fmt --check):**
    - **Purpose:** Validates that all smart contracts, test scripts, and deployment scripts perfectly follow the official Solidity Style Guide.

DevOps Intervention: Resolved multiple formatting breaks in test suites (e.g., FinalCoverage.t.sol and GovToken.t.sol) by standardizing single-line test block expressions into multi-line compliant blocks, allowing the check step to pass flawlessly.

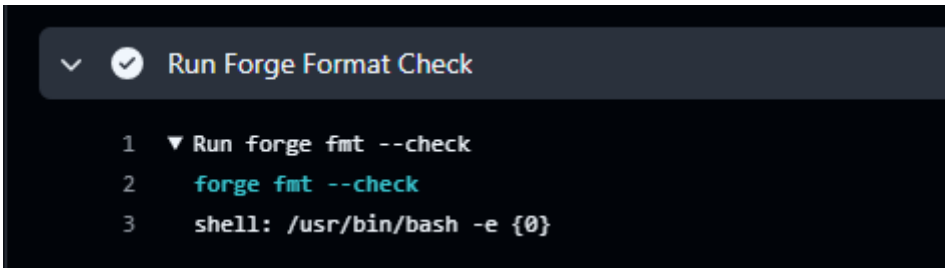


Figure 3: Validation gate confirming 100% compliance of Solidity source code with the official formatting style guide.

2. **Frontend Formatting (Run Prettier Check):**

- **Purpose:** Runs code layout evaluation on the client interface application components (.ts, .tsx, .json, .css).
- **DevOps Intervention:** Executed global repository cleanups using npx prettier --write to automatically align React codebase styling conventions before finalizing structural merges.

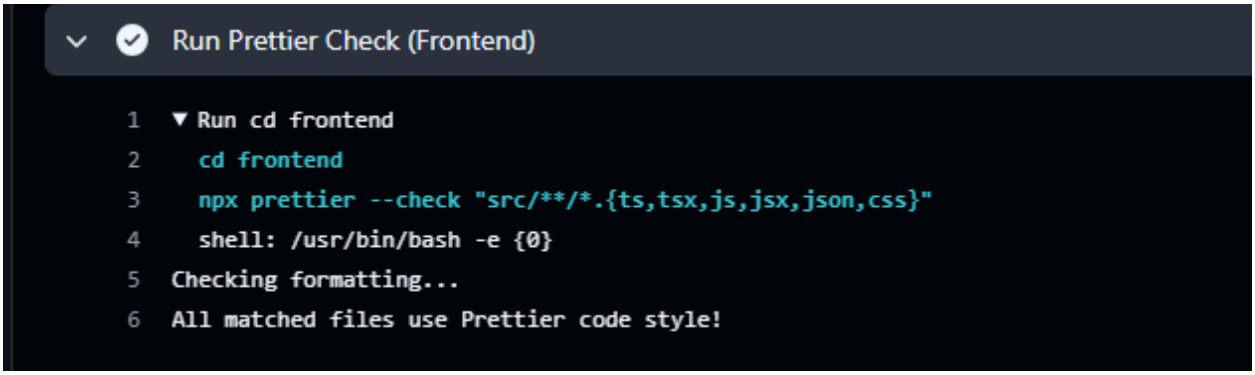


Figure 5: Automated Prettier code layout evaluation for React 19 and Wagmi modules.

4. Documentation Requirement: Architecture Decision Record (ADR)

- **Status:** Approved
- **Context:** With multiple developers working simultaneously on specialized features (Smart Contracts and Frontend UI), the project was vulnerable to breaking changes, non-standard styles, and hidden smart contract vulnerabilities entering production.
- **Options Considered:**
    1. *Option 1:* Manual verification by team members during code reviews.
    2. *Option 2:* Deploying an automated Git workflow that runs automated checks and strictly restricts merging unverified changes.
- **Decision: Option 2**. Implemented a multi-stage GitHub Actions runner configuration that combines syntax analysis, gas-optimized test validation, and static code auditing.
- **Consequences:** * *Positive:* Total elimination of human oversight during code updates; guaranteed delivery of fully formatted, compilable, and tested software components.
    - *Negative:* Slightly longer integration lifecycles, requiring roughly 2 minutes for processing cloud execution triggers per update.

5. Troubleshooting Log (DevOps Contribution Summary)

During the lifecycle of the system integration, I identified and manually resolved several infrastructure blockers:

- **Submodule & Nested Path Collision Fix:** Corrected a critical Git cache error where a duplicate nested directory (blockchain_final) was mistakenly classified as an unconfigured Git submodule, causing the remote checkout phase to crash. Cleaned the cache metadata and successfully stabilized dependency submodules (forge-std and OpenZeppelin libraries).
- **TypeScript Transport Incompatibility Fix:** Patched a compile-breaking type mismatch in src/config.ts related to Wagmi v3 and Viem provider structures running on React 19 by introducing rigid constant mappings and configuration adjustments.